A First Look at Object-Oriented Python

Engineering Computation with Python — Chapter 15

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why Objects?
2. Defining a Class
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. When to Use a Class — and When Not To
7. Summary

© Munim Zabidi

Table of Contents

1. Why Objects?
2. Defining a Class
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. When to Use a Class — and When Not To
7. Summary

Not a taught week

Roughly three lecture hours. This chapter is deliberately brief — Python's object system is large, and most of it is not needed for engineering computation.

By the end, you should be able to:

- Explain what a class and an object are, and how they differ.
- Define a class with `__init__` and store data in attributes.
- Write methods that operate on an object's own data via `self`.
- Use inheritance to share behaviour between related classes.
- Recognise that NumPy arrays and pandas DataFrames are themselves objects.

© Munim Zabidi

The Problem: Data and Functions, Kept Apart

The bench recording, handled the way earlier chapters taught it:

```
t = ...            # the time axis
v = ...            # the voltage samples
n_missing = count_missing(v)
v = repair_gaps(t, v)
tau = fit_time_constant(t, v)
```

Nothing is wrong with this …

… but nothing enforces it either. Nothing stops you passing the wrong time axis to `repair_gaps()`, or losing track of which of three recordings `v` currently is. The rules live only in the programmer's own carefulness.

The Idea: Bundle Data and Behaviour

An object bundles the time axis, the samples, and the operations that make sense on them into one thing:

```
rc = Signal(t, v, name="bench_rc voltage")
rc.repair()
tau = rc.time_constant()
```

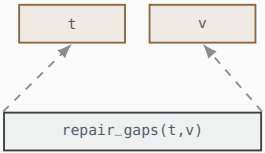
What changed

`rc.repair()` cannot be given the wrong time axis — it uses the one stored inside `rc`. The recording now carries its own operations with it.

Procedural vs. Object-Oriented

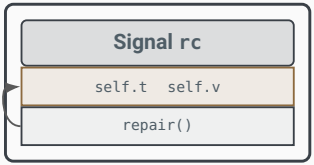
Procedural

Data and functions are separate. Any function can be applied to any data — including the wrong pairing.



Object-oriented

The object's boundary decides what may be applied to what. Methods can only reach their own data.



Two Words: Class and Object

Definitions

A class is the definition — a template describing what data an object holds and what it can do. An object (or instance) is one particular thing built from that template.

An analogy

The same relationship as between a circuit diagram and the boards built from it: one diagram, many boards, each with its own component values.

You have used objects since Chapter 6 without naming them. `a.mean()`: `a` is a `numpy.ndarray` object, `mean()` is one of its methods.

Table of Contents

1. Why Objects?
2. **Defining a Class**
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. When to Use a Class — and When Not To
7. Summary

The Smallest Useful Class

```python
class Resistor:
    def __init__(self, name, resistance):
        self.name = name                # store the name on this object
        self.resistance = resistance  # store the value on this object

r1 = Resistor("R1", 40.0)
r2 = Resistor("R2", 100.0)

print(r1.name, r1.resistance)
print(r2.name, r2.resistance)
```

```
R1 40.0    R2 100.0
```

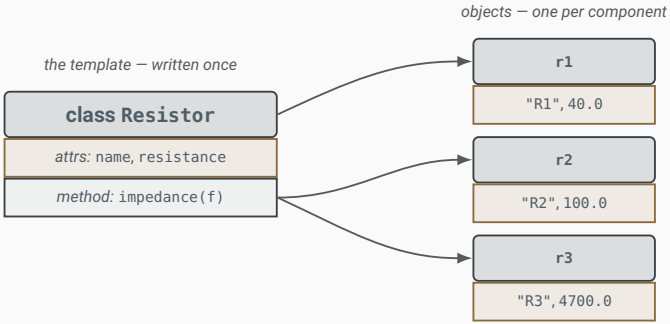
Three Pieces of Syntax

- `class Resistor:` begins the definition. Class names are capitalised by convention — that is how you can tell `Resistor(...)` creates an object rather than calling an ordinary function.
- `__init__` is a special method Python calls automatically whenever a new object is created, to set up its starting data. You never call it directly.
- `self` refers to the particular object being worked on. It is the first parameter of every method, and Python supplies it automatically — which is why `Resistor("R1", 40.0)` passes two arguments to a method written with three parameters.

Attributes

Values stored with `self` are attributes, read back with a dot: `r1.resistance`. Each object keeps its own copy.

One Class, Many Objects



Each object gets its own copy of the data, but they share the one definition of `impedance()`.

Table of Contents

1. Why Objects?
2. Defining a Class
3. **Adding Methods**
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. When to Use a Class — and When Not To
7. Summary

A Method: a Function Defined Inside a Class

A class that only stores data is little better than a dictionary — the point of an object is that it also carries the operations valid on that data:

```
class Resistor:
    def __init__(self, name, resistance):
        self.name = name
        self.resistance = resistance

    def impedance(self, f):
        # A resistor's impedance is real, independent of frequency
        return complex(self.resistance, 0)

r1 = Resistor("R1", 40.0)
print(r1.impedance(1000))
```

```
(40+0j)
```

Reading the Method

The method takes `self` first, then its own arguments. Inside, it reaches the object's stored data through `self.resistance` — no separate resistance parameter is needed, because the object already knows its own value.

Calling it

`r1.impedance(1000)`: the object before the dot becomes `self`.

Table of Contents

1 Why Objects?

2 Defining a Class

3 Adding Methods

4 **Inheritance: Sharing Behaviour**

5 EE Focus: Wrapping the Bench Recording

6 When to Use a Class — and When Not To

7 Summary

Resistors, Inductors, Capacitors: Mostly the Same

Each has a name and a value, each can report an impedance — but each computes it differently (Chapter 8).

Rather than three unrelated classes with duplicated code . . .

. . . one class can be defined as a specialised version of another. This is inheritance.

A Shared Base Class

```python
class Component:
    """Shared behaviour for any two-terminal component."""
    def __init__(self, name, value):
        self.name = name
        self.value = value

    def impedance(self, f):
        raise NotImplementedError("Each type defines its own impedance")

    def describe(self, f):
        Z = self.impedance(f)
        return f"{self.name}: Z = {Z:.2f} ohm, |Z| = {abs(Z):.2f} ohm"
```

Three Subclasses, One Line Each

```
class Resistor(Component):
    def impedance(self, f):
        return complex(self.value, 0)          # Z = R

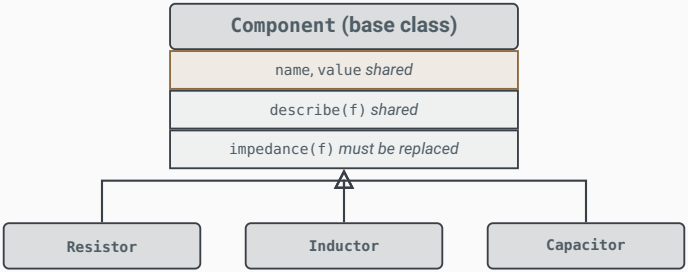
class Inductor(Component):
    def impedance(self, f):
        return 1j * 2 * np.pi * f * self.value  # Z = +j*w*L

class Capacitor(Component):
    def impedance(self, f):
        return -1j / (2 * np.pi * f * self.value)  # Z = -j/(w*C)
```

What `class Resistor(Component)` declares

A `Resistor` is a `Component`: it automatically gains `__init__` and `describe()` without repeating them. Replacing an inherited method is called overriding.

What Is Shared, What Is Replaced



All three subclasses inherit `name`, `value`, and `describe()` unchanged; each overrides `impedance()` with its own physics.

The Payoff: Uniform Treatment

```
f = 60.0
circuit = [Resistor("R", 40.0), Inductor("L", 0.2), Capacitor("C", 50e-6)]

for c in circuit:
    print(c.describe(f))

Z_total = sum(c.impedance(f) for c in circuit)  # Series impedances add
print(f"Series total: Z = {Z_total:.2f} ohm")
```

```
R: Z = 40.00+0.00j ohm, |Z| = 40.00 ohm
L: Z = 0.00+75.40j ohm, |Z| = 75.40 ohm
C: Z = -0.00-53.05j ohm, |Z| = 53.05 ohm

Series total: Z = 40.00+22.35j ohm
```

Check

These are the same numbers computed in Chapter 8 for the same series RLC circuit: $Z = 40 + 22.35j\ \Omega$, magnitude $45.82\ \Omega$, phase $29.2°$.

The physics has not changed and the arithmetic has not changed. What changed is the organisation: the loop over `circuit` does not know or care which kind of component it holds — it simply asks each one for its impedance.

Adding a fourth component type would require no change to that loop at all.

© Muním Zabidi

Table of Contents

1. Why Objects?
2. Defining a Class
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. **EE Focus: Wrapping the Bench Recording**
6. When to Use a Class — and When Not To
7. Summary

The second natural use of a class in this book: the recurring bench dataset. A Signal object can hold a time axis and its samples, and carry the operations developed across earlier chapters (arrays, error handling, curve fitting).

© Muním Zabidi

The Signal Class

```python
class Signal:
    """A time series of measurements, with operations valid on it."""
    def __init__(self, t, v, name="signal"):
        self.t = np.asarray(t, dtype=float)
        self.v = np.asarray(v, dtype=float)
        self.name = name

    def n_missing(self):
        return int(np.isnan(self.v).sum())

    def repair(self):
        gaps = np.isnan(self.v)
        if gaps.any():
            self.v[gaps] = np.interp(self.t[gaps], self.t[~gaps], self.v[~gaps])
        return self

    def time_constant(self):
        return -1 / np.polyfit(self.t, np.log(self.v), 1)[0]
```

```
t = np.arange(0, 2.001, 0.02)
v = 12.0 * np.exp(-t / 0.47)
v[40:43] = np.nan          # the three samples the logger lost near t=0.8 s

rc = Signal(t, v, name="bench_rc voltage")
print(rc.summary())

rc.repair()
print(rc.summary())
print(f"Measured tau: {rc.time_constant():.4f} s")
```

```
bench_rc voltage: 101 samples, 3 missing, v from 12.00 to 0.17 V
bench_rc voltage: 101 samples, 0 missing, v from 12.00 to 0.17 V
Measured tau: 0.4700 s
```

© Muním Zabidi

When Does This Earn Its Keep?

Everything this class does was already possible with plain functions — for a single recording, the plain version is perfectly reasonable.

The organisation earns its keep when . . .

. . . there are several recordings: voltage, current, and temperature from the same logger, or the same experiment repeated at three temperatures. Each becomes its own `Signal`, and no call can accidentally pair one recording's samples with another's time axis.

A Caution About Mutation

`repair()` modifies `self.v` in place, so the original samples are overwritten.

A genuine design decision, not an accident of syntax

Convenient — but the raw measurement is gone. Chapter 11 argued that repairs should be documented, not silent.

Where the unrepaired data must be preserved, the method should instead return a *new* `Signal` and leave the original untouched.

Table of Contents

1. Why Objects?
2. Defining a Class
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. **When to Use a Class — and When Not To**
7. Summary

A Tool With a Specific Purpose

OOP is not a better way to write everything. For the engineering calculations in this book, plain functions and NumPy arrays are usually the clearer choice.

A class is worth defining when …

- Several pieces of data always belong together.
- A set of operations only makes sense on that particular data.
- You need several independent copies of the same kind of thing.
- Related kinds of thing share most of their behaviour and differ in a little of it.

© Munim Zabidi

A Plain Function Is Usually Better

When the operation is a self-contained calculation that takes inputs and returns an output — which describes most of what an engineer computes.

Converting units, fitting a curve, integrating a signal: none of these need a class.

Why learn objects anyway?

The libraries central to engineering computation are built from them. Every `arr.mean()`, `df.groupby()`, or `ax.set_xlabel()` is a method call on an object. Knowing this turns documentation from a list of incantations into something you can reason about.

Table of Contents

1. Why Objects?
2. Defining a Class
3. Adding Methods
4. Inheritance: Sharing Behaviour
5. EE Focus: Wrapping the Bench Recording
6. When to Use a Class — and When Not To
7. **Summary**

Summary

- A class is a template; an object is one instance built from it. NumPy arrays and pandas DataFrames are objects — hence `a.mean()`, not `mean(a)`.
- `__init__` runs automatically on creation and sets up starting data. Never called directly.
- `self` refers to the particular object a method is working on. `self.something` values are attributes.
- A method is a function defined inside a class, reaching the object's data through `self`.
- Inheritance lets a class reuse another's behaviour and replace only what differs (overriding).
- Objects organise code; they do not change the mathematics.
- Use a class when data and operations genuinely belong together. Use a plain function otherwise — which covers most engineering computation.

Where This Is Going

This chapter is enrichment — the core path through the book does not require writing your own classes.

- You will keep using objects constantly: every NumPy array, every pandas DataFrame, every Matplotlib axis.
- Chapter 16's mini-projects return to plain functions and arrays, the everyday tools of this course.
- If a future project grows large enough that several recordings must travel with their own rules, you now know the tool that organises it.

Next: Engineering Mini-Projects

© Muním Zabidi